

**Planeación del software: sistema de chat empresarial cliente-servidor en Go con
autenticación FastAPI**

Cesar Bolivar Arciniegas Mejia

[Nombres de los demás integrantes del grupo]

Universidad Internacional del Ecuador (UIDE)

Programación Orientada a Objetos, 03-SIN-A

Proyecto Integrador, Fase 1-2: Etapa 1

[Nombre del docente]

5 de octubre de 2026

Justificación del sistema seleccionado

De las cuatro opciones que propone el proyecto (correos internos, chat empresarial, gestión de imágenes y gestión de videos), elegimos el chat empresarial. La consigna pide que la opción resuelva un problema organizacional, permita operaciones cliente-servidor reales y produzca datos medibles para el componente estadístico. El chat cumple las tres condiciones sin exigirnos piezas que no podamos terminar en el tiempo del curso.

Necesidad organizacional

En una empresa pequeña o mediana, buena parte del trabajo diario se resuelve con preguntas cortas entre compañeros: quién tiene la última versión de un documento, si el cliente ya respondió, qué se acordó en la reunión de ayer. Cuando la organización no ofrece un canal propio, esas conversaciones se reparten entre aplicaciones de mensajería personales, con cuentas que la empresa no administra.

Eso trae problemas concretos. Nadie sabe con certeza quién tiene acceso a lo que se comparte en un grupo. El historial vive en el teléfono de cada persona y se pierde cuando alguien cambia de equipo o deja la empresa. Y la dirección no puede responder preguntas básicas, como cuántas personas usan el canal a diario o en qué horas se concentra la actividad. Un chat interno, con cuentas creadas por la propia organización y datos guardados en su servidor, ataca esos puntos a la vez.

Frente a las otras opciones, el chat produce de forma natural lo que pide el componente estadístico: cada mensaje, inicio de sesión y consulta es un evento con usuario, tipo y hora. Las opciones de imágenes y videos agregan el manejo de archivos grandes, algo que nos habría quitado tiempo de lo que esta etapa evalúa.

Usuarios que se beneficiarán

Los beneficiarios directos son los empleados, que ganan un único lugar para escribirse con sus compañeros y volver a leer lo conversado. El personal de TI o de administración se beneficia porque crea y desactiva cuentas, controla las sesiones abiertas y decide quién entra a cada sala. La dirección, por su parte, obtiene indicadores de uso sin tener que pedírselos a nadie.

Operaciones principales

El sistema permitirá iniciar y cerrar sesión, enviar mensajes de texto a salas y a otros usuarios, recibirlos en tiempo real, consultar el historial, ver quién está conectado, administrar usuarios, salas y sesiones, y consultar estadísticas de uso. Cada una de estas operaciones aparece más adelante como caso de uso.

Información que administrará

El sistema maneja cinco tipos de información, que resumimos en la Tabla 1 junto con el componente que la guarda. Las cuentas y las sesiones quedan en el servicio de autenticación; las salas y los mensajes, en el servidor de chat; y los eventos de uso se registran en ambos, cada uno con lo que ve.

Tabla 1

Información que administra el sistema

Entidad	Datos principales	Componente que la guarda
Usuario	Identificador, nombre de usuario, correo, hash de la contraseña, rol, estado (activo o inactivo)	Servicio de autenticación (FastAPI)
Sesión	Identificador, usuario, token, hora de inicio, hora de vencimiento, hora de cierre	Servicio de autenticación (FastAPI)
Sala	Identificador, nombre, descripción, creador, estado, miembros	Servidor de chat (Go)
Mensaje	Identificador, remitente, sala o destinatario, texto, fecha y hora asignadas por el servidor	Servidor de chat (Go)
Evento de uso	Tipo de evento, usuario, fecha y hora, duración o latencia cuando aplica	Ambos, según el evento

Importancia de la arquitectura cliente-servidor

Un chat necesita un punto central. Si cada cliente hablara directamente con los demás, no habría un historial común, nadie controlaría quién entra y sería imposible medir el uso. Por eso el servidor de chat, escrito en Go, recibe todos los mensajes, los guarda y los reenvía. Go atiende bien muchas conexiones a la vez gracias a sus goroutines y canales (Donovan y Kernighan, 2015), y para mantener abierta la conexión entre cliente y servidor usaremos WebSocket, un protocolo que permite comunicación en ambos sentidos sobre una sola conexión TCP (Fette y Melnikov, 2011).

La autenticación queda fuera del servidor de chat a propósito. La delegamos en un servicio FastAPI, un marco de Python para construir APIs HTTP (Ramírez, s. f.), que guarda las cuentas, verifica contraseñas y emite tokens JWT, un formato compacto que lleva firma y fecha de vencimiento (Jones et al., 2015). Así el servidor de chat nunca toca una contraseña, y el mismo servicio podría atender a otras aplicaciones de la empresa.

En la práctica, el recorrido es este. El cliente envía usuario y contraseña al servicio FastAPI y recibe un token. Con ese token abre la conexión con el servidor de chat, que antes de aceptarla consulta al servicio si el token sigue vigente. Desde ahí, cada mensaje viaja del cliente al servidor, se guarda y se reenvía a los demás conectados. Cada paso es una petición real entre procesos distintos, y cada una deja un registro que después sirve para las estadísticas.

Alcance preliminar

El alcance mínimo incluye autenticación con roles, mensajes en salas y directos, historial, lista de usuarios conectados, control de sesiones y estadísticas básicas de uso. Dejamos fuera, por ahora, los archivos adjuntos, las llamadas, el cifrado de extremo a extremo, las notificaciones push y las aplicaciones móviles.

Objetivos

Redactamos los objetivos con verbos en infinitivo y de manera que, al cierre del proyecto, podamos comprobar si se cumplieron.

Objetivo general

Desarrollar una aplicación de chat empresarial cliente-servidor en Go, integrada con un servicio de autenticación en FastAPI, que permita a usuarios autenticados intercambiar mensajes de texto y consultar su historial, y que registre datos de uso para su análisis estadístico.

Objetivos específicos

OE1. Funcionalidad. Implementar el envío y la recepción en tiempo real de mensajes de texto en salas y conversaciones directas, con historial consultable.

OE2. Integración. Integrar el cliente y el servidor de chat en Go con el servicio FastAPI, de modo que toda conexión requiera un token válido.

OE3. Seguridad. Controlar el acceso mediante roles, vencimiento de tokens y gestión de sesiones activas.

OE4. Mantenibilidad. Organizar el código en módulos separados por responsabilidad, con pruebas unitarias y un repositorio Git donde cada integrante deje commits propios.

OE5. Validación. Registrar los eventos de uso y calcular indicadores, comprobándolos con una prueba de carga básica de al menos 20 usuarios simulados.

Actores y usuarios

Identificamos tres actores: dos personas que trabajan con sesión (el usuario y el administrador) y un actor de sistema, el servicio de autenticación. El administrador hereda todo lo que puede hacer el usuario. La Tabla 2 resume a cada uno.

Tabla 2*Actores del sistema*

Actor	Descripción	Responsabilidades	Operaciones permitidas
Usuario	Empleado con cuenta activa y rol de usuario. Antes de iniciar sesión solo puede iniciar sesión.	Mantener su contraseña en reserva y cerrar sesión al terminar.	Iniciar y cerrar sesión; unirse a salas; enviar mensajes a salas y directos; consultar historial; ver usuarios conectados.
Administrador	Persona de TI o de la jefatura con rol de administrador. Hereda las operaciones del usuario.	Crear y mantener cuentas y salas, supervisar las sesiones y revisar los indicadores de uso.	Gestionar usuarios (crear, activar o desactivar, asignar rol); gestionar salas; consultar y cerrar sesiones; consultar estadísticas.
Servicio de autenticación (FastAPI)	Actor de sistema, externo al servidor de chat.	Guardar cuentas, verificar credenciales, emitir y validar tokens, y registrar los eventos de autenticación.	Validar credenciales; emitir y validar tokens; invalidar sesiones; reportar eventos de autenticación.

Requisitos

Separamos los requisitos en funcionales, que dicen qué hace el sistema, y no funcionales, que fijan con qué calidad lo hace (Sommerville, 2016). Cada uno tiene un código para poder seguirlo hasta los casos de uso y los criterios de aceptación, y los redactamos de forma que se puedan verificar (ISO/IEC/IEEE, 2018).

Requisitos funcionales

Tabla 3
Requisitos funcionales

Código	Requisito	Prioridad
RF-01	El administrador crea cuentas indicando nombre de usuario, correo, rol y contraseña inicial.	Alta
RF-02	El usuario inicia sesión con usuario y contraseña, y el sistema le entrega un token con vencimiento.	Alta
RF-03	El usuario cierra sesión y el sistema invalida su token.	Alta
RF-04	El sistema vence las sesiones inactivas, y el administrador puede listar y cerrar sesiones activas.	Alta
RF-05	El usuario envía mensajes de texto a una sala de la que es miembro.	Alta
RF-06	El usuario envía mensajes directos a otro usuario.	Media
RF-07	El sistema entrega los mensajes en tiempo real a los usuarios conectados que correspondan.	Alta
RF-08	El usuario consulta el historial paginado de una sala o de una conversación directa.	Alta
RF-09	El administrador crea y archiva salas, y los usuarios se unen a las salas disponibles.	Media
RF-10	El sistema muestra qué usuarios están conectados.	Media
RF-11	El administrador consulta estadísticas de uso por rango de fechas.	Alta
RF-12	El administrador activa o desactiva cuentas y asigna roles.	Media
RF-13	El sistema registra los eventos de uso (sesiones, mensajes, consultas) con fecha y hora del servidor.	Alta

Requisitos no funcionales

Tabla 4
Requisitos no funcionales

Código	Categoría	Requisito
RNF-01	Seguridad	Las contraseñas se guardan con hash y sal (por ejemplo, bcrypt), nunca en texto plano. Los tokens van firmados y vencen. En el despliegue final, la comunicación usará TLS.
RNF-02	Rendimiento	Con 20 usuarios conectados en red local, al menos el 90 % de los mensajes se entrega en menos de 1 segundo.
RNF-03	Tolerancia a fallos	Si el cliente pierde la conexión, intenta reconectarse solo y conserva el texto que no alcanzó a enviar.
RNF-04	Mantenibilidad	El código se separa en paquetes por responsabilidad (autenticación, mensajería, almacenamiento, cliente) y la lógica principal tiene pruebas unitarias.
RNF-05	Portabilidad	El cliente se compila y ejecuta en Linux y en Windows.
RNF-06	Usabilidad	Los mensajes de error indican qué falló y qué hacer; las operaciones principales se alcanzan en tres pasos o menos desde el menú.
RNF-07	Auditoría	Cada evento relevante deja un registro con usuario, tipo de evento y hora del servidor.
RNF-08	Integridad	Un mensaje no se modifica después de guardado, y su hora la asigna el servidor, no el cliente.

Casos de uso

Modelamos el sistema con diagramas de casos de uso de UML (Object Management Group [OMG], 2017) y describimos los casos más importantes con la plantilla de Cockburn (2001), que separa el flujo básico de los flujos alternativos y fija precondiciones y postcondiciones. La Tabla 5 lista los once casos de uso del sistema.

Tabla 5

Catálogo de casos de uso

Código	Caso de uso	Actor principal	Requisitos
CU-01	Iniciar sesión	Usuario	RF-02, RF-13
CU-02	Cerrar sesión	Usuario	RF-03
CU-03	Enviar mensaje a una sala	Usuario	RF-05, RF-07, RF-13
CU-04	Enviar mensaje directo	Usuario	RF-06, RF-07, RF-13
CU-05	Consultar historial	Usuario	RF-08
CU-06	Ver usuarios conectados	Usuario	RF-10
CU-07	Gestionar usuarios	Administrador	RF-01, RF-12
CU-08	Gestionar salas	Administrador	RF-09
CU-09	Controlar sesiones	Administrador	RF-04
CU-10	Consultar estadísticas de uso	Administrador	RF-11, RF-13
CU-11	Unirse a una sala	Usuario	RF-09

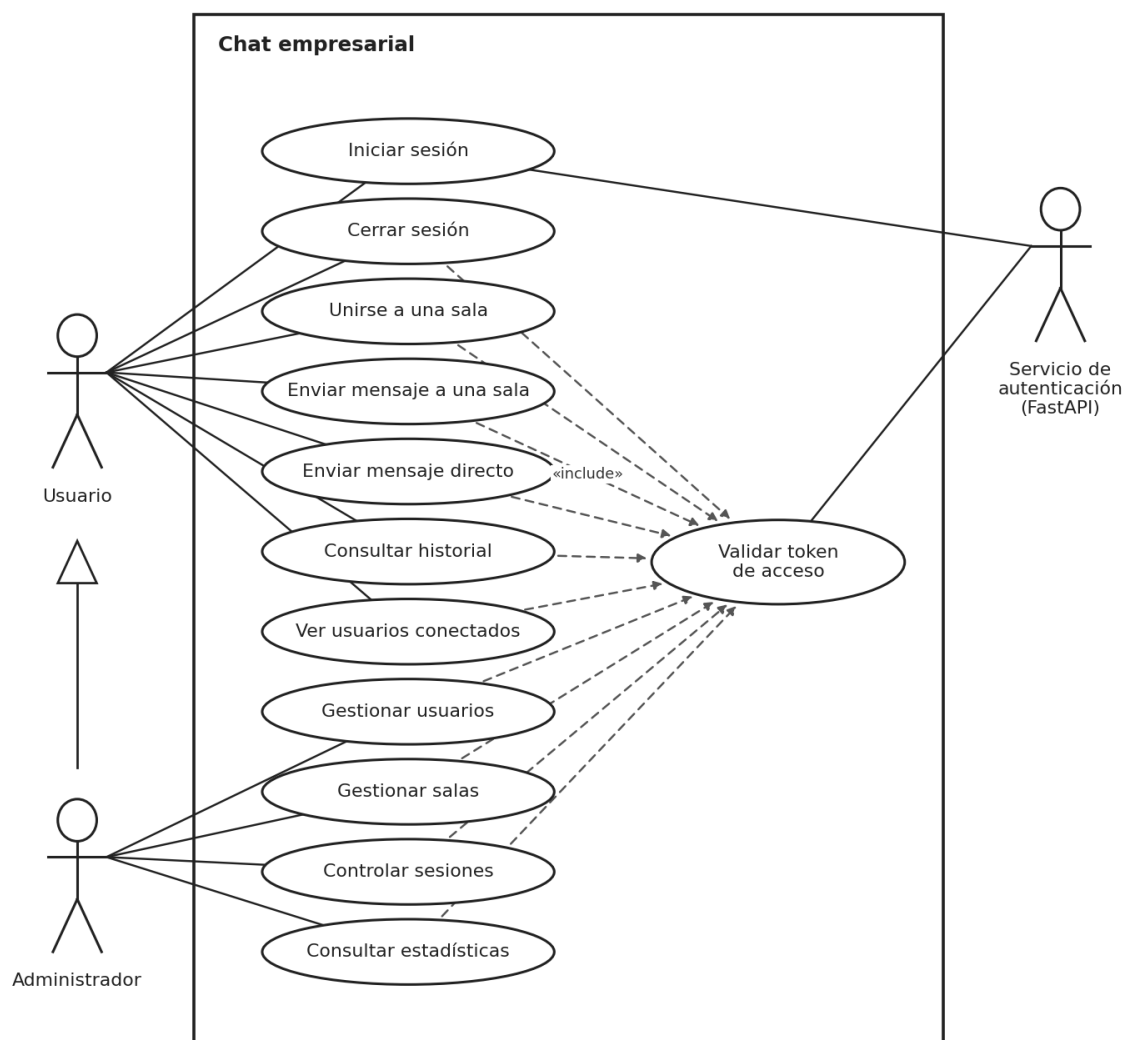
Nota. Los casos incluidos o extendidos que aparecen solo en los diagramas (validar token, validar credenciales, registrar intento fallido, expirar sesión, filtrar historial, calcular indicadores y exportar reporte) forman parte del caso que los usa y no tienen código propio.

Diagramas de casos de uso

La Figura 1 es el diagrama general. Muestra a los tres actores, las operaciones principales y la autenticación que exige el sistema: todo caso de uso, salvo iniciar sesión, incluye la validación del token contra el servicio FastAPI.

Figura 1

Diagrama general de casos de uso del sistema de chat empresarial

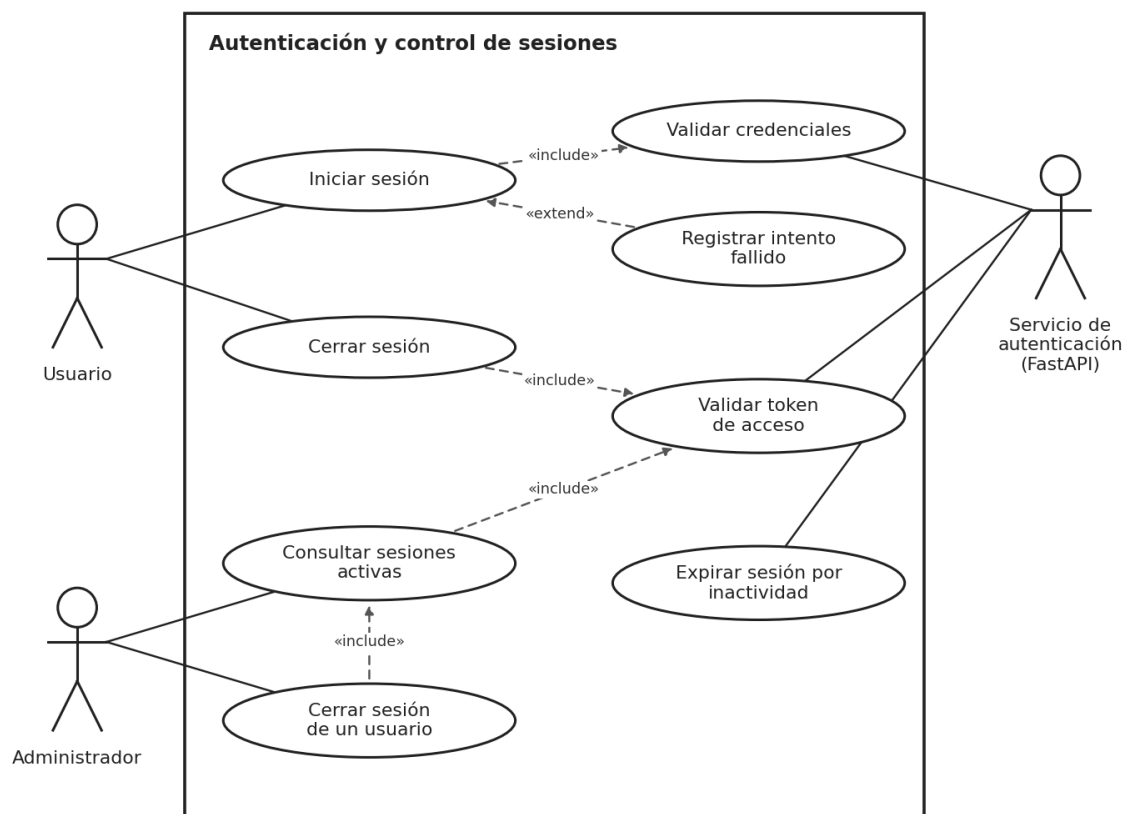


Nota. Las flechas punteadas son relaciones «include» hacia «Validar token de acceso». El administrador hereda las operaciones del usuario.

La Figura 2 detalla la autenticación y el control de sesiones. Ahí aparecen los casos que hacen medible la seguridad, como los intentos fallidos de inicio de sesión y los vencimientos por inactividad.

Figura 2

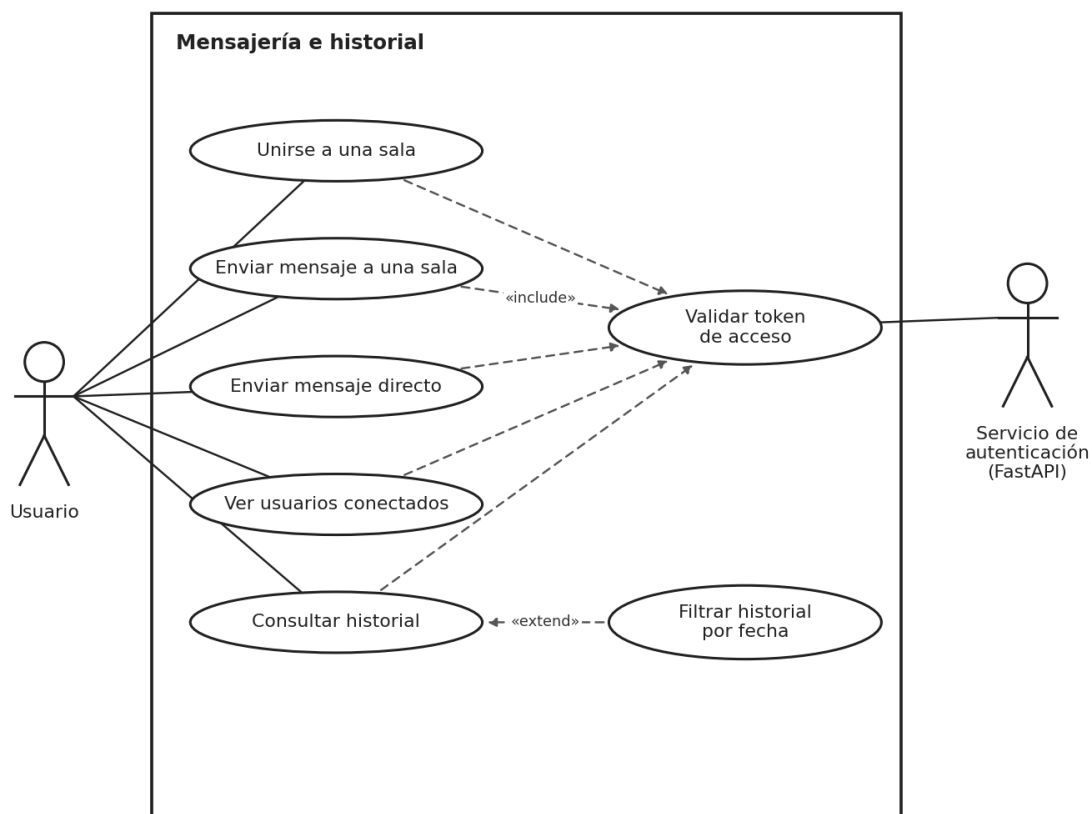
Diagrama de casos de uso de autenticación y control de sesiones



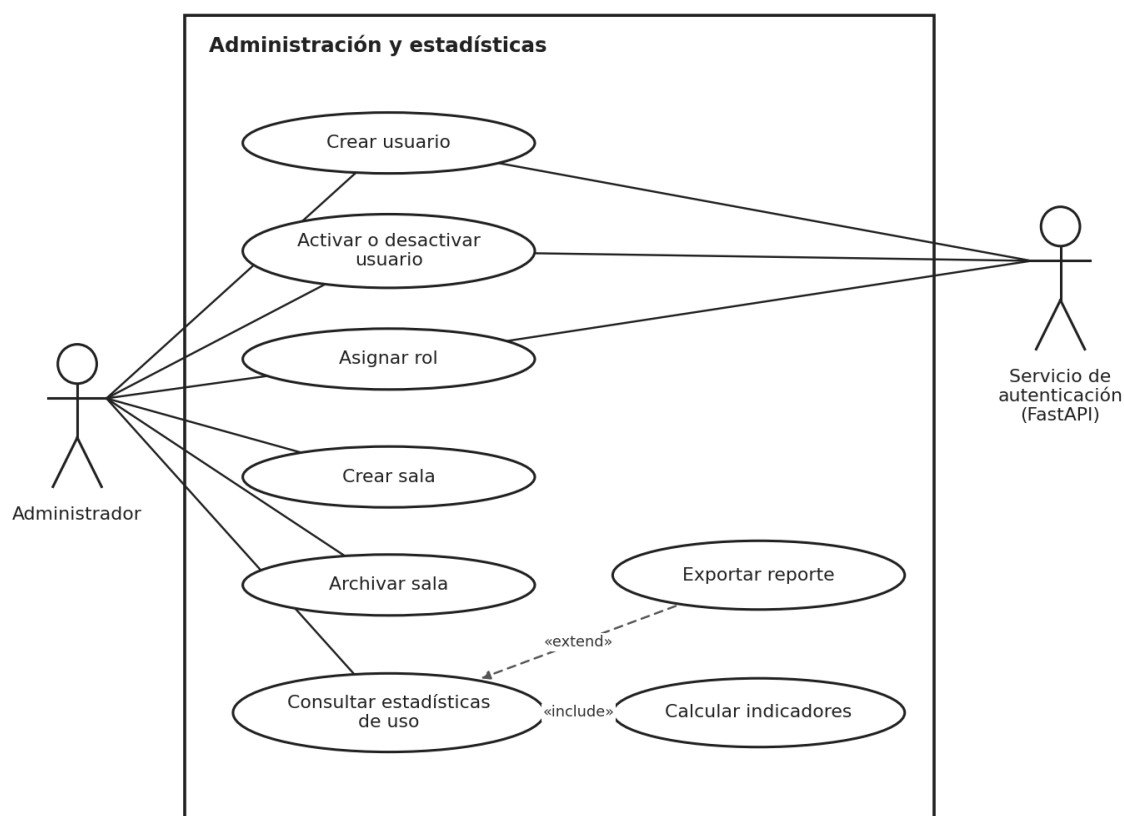
La Figura 3 cubre la mensajería. El historial admite un filtro por fecha, que modelamos como extensión porque el usuario puede consultar el historial sin usarlo.

Figura 3

Diagrama de casos de uso de mensajería e historial



La Figura 4 reúne lo que solo hace el administrador. Todos estos casos requieren un token válido con rol de administrador, validado igual que en la Figura 1.

Figura 4*Diagrama de casos de uso de administración y estadísticas***Descripción de los casos de uso principales**

Describimos cuatro casos: iniciar sesión, enviar mensaje a una sala, consultar historial y consultar estadísticas. Los elegimos porque juntos cubren el acceso, la mensajería, la lectura de lo guardado y la parte estadística.

Tabla 6*CU-01. Iniciar sesión*

Campo	Detalle
Actores	Usuario (principal). Servicio de autenticación FastAPI (secundario).
Descripción	El usuario ingresa sus credenciales, el servicio las verifica y le entrega un token con vencimiento. Con ese token el cliente abre la conexión con el servidor de chat.
Precondiciones	La cuenta existe y está activa. El cliente, el servidor de chat y el servicio de autenticación están en ejecución y son alcanzables.
Flujo básico	1. El usuario abre el cliente Go e ingresa nombre de usuario y contraseña. 2. El cliente envía las credenciales al endpoint de inicio de sesión del servicio FastAPI. 3. FastAPI busca la cuenta y compara la contraseña con el hash guardado. 4. FastAPI crea la sesión, genera un token firmado con vencimiento y lo devuelve. 5. El cliente guarda el token en memoria y abre la conexión WebSocket con el servidor de chat, enviando el token. 6. El servidor de chat consulta a FastAPI si el token es válido y registra al usuario como conectado. 7. El cliente muestra la lista de salas del usuario.
Flujos alternativos	3a. Credenciales incorrectas: FastAPI responde 401, registra el intento fallido y el cliente muestra el error. Tras varios intentos seguidos, la cuenta se bloquea un tiempo (el número se define en el diseño). 3b. Cuenta desactivada: FastAPI responde 403 y el cliente lo informa. 5a. Servicio de autenticación no disponible: el cliente muestra el error y permite reintentar. 6a. Token inválido: el servidor cierra la conexión y el cliente vuelve a la pantalla de inicio.
Postcondiciones	Existe una sesión activa con su hora de inicio. El usuario figura como conectado. Queda guardado el evento de inicio de sesión, exitoso o fallido.
Datos estadísticos	Inicios de sesión exitosos y fallidos por día; tiempo de respuesta de la autenticación, en milisegundos.

Tabla 7*CU-03. Enviar mensaje a una sala*

Campo	Detalle
Actores	Usuario (principal). Servicio de autenticación FastAPI (secundario, valida el token).
Descripción	El usuario escribe un texto y lo envía a una sala de la que es miembro. El servidor lo guarda y lo reenvía a los usuarios conectados de esa sala.
Precondiciones	Hay una sesión activa con token vigente. El usuario es miembro de la sala y la sala está activa.
Flujo básico	1. El usuario elige una sala y escribe su mensaje. 2. El cliente comprueba que el texto no esté vacío ni pase de 1 000 caracteres. 3. El cliente envía el mensaje por la conexión abierta. 4. El servidor confirma que el token sigue vigente. 5. El servidor asigna un identificador y la hora del servidor, y guarda el mensaje. 6. El servidor reenvía el mensaje a todos los conectados de la sala, incluido el remitente. 7. Los clientes muestran el mensaje con el nombre del remitente y la hora.
Flujos alternativos	2a. Mensaje vacío o demasiado largo: el cliente no lo envía y avisa al usuario. 3a. Se perdió la conexión: el cliente intenta reconectarse y conserva el texto sin enviar. 4a. Token vencido o sesión cerrada: el servidor rechaza el mensaje, cierra la conexión y el cliente pide iniciar sesión de nuevo. 5a. Falla el guardado: el servidor no reenvía nada y avisa al remitente para que reintente. 6a. Un miembro está desconectado: no recibe el mensaje en vivo y lo verá en el historial al volver.
Postcondiciones	El mensaje queda guardado con su fecha y hora y disponible en el historial. Se registra el evento de envío.
Datos estadísticos	Mensajes por sala y por día; latencia entre la recepción y la entrega, en milisegundos; mensajes rechazados.

Tabla 8*CU-05. Consultar historial*

Campo	Detalle
Actores	Usuario (principal). Servicio de autenticación FastAPI (secundario, valida el token).
Descripción	El usuario lee los mensajes anteriores de una sala o de una conversación directa, de los más recientes a los más antiguos.
Precondiciones	Hay una sesión activa. El usuario es miembro de la sala o participante de la conversación.
Flujo básico	1. El usuario abre una sala o una conversación. 2. El cliente pide al servidor los últimos 50 mensajes. 3. El servidor valida el token y comprueba que el usuario pertenece a la sala. 4. El servidor consulta el almacenamiento y devuelve los mensajes ordenados por fecha. 5. El cliente los muestra en orden cronológico. 6. Si el usuario sigue subiendo, el cliente pide la página anterior hasta que no queden más.
Flujos alternativos	2a. Filtro por fecha: el usuario indica un rango y el servidor devuelve solo esos mensajes. 3a. El usuario no pertenece a la sala: el servidor responde 403 y el cliente lo informa. 4a. No hay mensajes: el cliente muestra «Aún no hay mensajes». 4b. Falla la consulta: el cliente muestra el error y permite reintentar.
Postcondiciones	Ningún mensaje se modifica. Se registra la consulta.
Datos estadísticos	Consultas de historial por usuario y por día; tiempo de respuesta de la consulta; cantidad de mensajes devueltos.

Tabla 9*CU-10. Consultar estadísticas de uso*

Campo	Detalle
Actores	Administrador (principal). Servicio de autenticación FastAPI (secundario).
Descripción	El administrador consulta indicadores de uso del chat para un rango de fechas.
Precondiciones	Hay una sesión activa con rol de administrador. Existen eventos registrados.
Flujo básico	1. El administrador elige la opción de estadísticas. 2. El cliente pide al servidor los indicadores del rango elegido (por defecto, los últimos 7 días). 3. El servidor valida el token y el rol. 4. El servidor agrupa sus eventos de mensajería y pide a FastAPI los de autenticación. 5. El servidor calcula mensajes por día y por sala, usuarios activos, inicios de sesión exitosos y fallidos, y latencia promedio. 6. El cliente muestra el resumen en tablas.
Flujos alternativos	2a. Rango inválido (la fecha inicial es posterior a la final): el cliente avisa y no envía la petición. 3a. El usuario no es administrador: el servidor responde 403. 5a. No hay datos en el rango: el cliente muestra ceros y un aviso. 6a. Exportar reporte: el administrador guarda el resumen en un archivo CSV.
Postcondiciones	Ningún dato se modifica. Se registra la consulta de estadísticas.
Datos estadísticos	Número de consultas de estadísticas; tiempo que tarda en generarse el reporte.

Matriz de trazabilidad inicial

La Tabla 10 une cada necesidad con su objetivo, sus requisitos, sus casos de uso y un criterio de aceptación que se pueda comprobar. La revisamos en los dos sentidos: cada necesidad llega al menos a un caso de uso, y todos los requisitos funcionales y todos los casos de uso aparecen al menos en una fila.

Tabla 10*Matriz de trazabilidad inicial*

Necesidad identificada	Objetivo	Requisitos	Casos de uso	Criterio de aceptación
N-01. Los empleados necesitan un canal interno único para comunicarse.	OE1	RF-05, RF-06, RF-07, RNF-02, RNF-03	CU-03, CU-04, CU-11	Un mensaje enviado a una sala llega a todos los usuarios conectados de esa sala. En la prueba con 20 usuarios, al menos el 90 % llega en menos de 1 s.
N-02. Solo el personal autorizado debe acceder.	OE2, OE3	RF-02, RNF-01	CU-01	Con credenciales válidas se entrega un token con vencimiento. Con credenciales inválidas el servicio responde 401 y no se abre la conexión.
N-02. Solo el personal autorizado debe acceder.	OE3	RF-03, RF-04	CU-02, CU-09	Después de cerrar sesión, o de que el administrador la cierre, el servidor rechaza el token anterior.
N-03. Las conversaciones deben conservarse y poder consultarse.	OE1	RF-08, RNF-08	CU-05	Al abrir una sala se muestran los últimos 50 mensajes en orden cronológico, incluidos los enviados antes de que el usuario se conectara.
N-04. Hay que saber quién está conectado y controlar las sesiones abiertas.	OE3	RF-04, RF-10	CU-06, CU-09	La lista de conectados cambia en menos de 5 s tras una conexión o desconexión. El administrador ve las sesiones activas y puede cerrar una.
N-05. La dirección necesita medir el uso del chat.	OE5	RF-11, RF-13, RNF-07	CU-10	El reporte muestra mensajes por día y por sala, usuarios activos e inicios de sesión. Las cifras coinciden con las de una prueba controlada.
N-06. Alguien debe administrar cuentas y salas.	OE1, OE3	RF-01, RF-09, RF-12	CU-07, CU-08	Un usuario creado por el administrador puede iniciar sesión. Una sala creada aparece en la lista de salas de sus miembros.
N-07. El equipo debe poder mantener y probar el sistema.	OE4	RNF-04, RNF-05, RNF-06	Todos	Los módulos de autenticación y mensajería tienen pruebas unitarias y el historial Git muestra commits de cada integrante.

Nota. N = necesidad; OE = objetivo específico; RF y RNF = requisitos funcionales y no funcionales; CU = caso de uso.

Componentes y decisiones de arquitectura

La Tabla 11 separa lo que hace cada componente. El cliente en Go es la cara visible del sistema, el servidor de chat concentra la mensajería, y el servicio FastAPI es el único que conoce las contraseñas. Dividimos así para que un cambio en la autenticación no obligue a tocar la mensajería, y al revés.

Tabla 11*Componentes del sistema y casos de uso que atienden*

Componente	Tecnología	Responsabilidad	Casos de uso
Cliente	Go, cliente de consola	Pedir credenciales, guardar el token en memoria, abrir la conexión, mostrar mensajes y estadísticas, y reconectarse si se cae la red.	Interfaz de todos los casos de uso
Servidor de chat	Go, WebSocket	Aceptar conexiones, validar el token con FastAPI, guardar y reenviar mensajes, mantener salas y presencia, registrar eventos de mensajería y calcular indicadores.	CU-03 a CU-06, CU-08, CU-10, CU-11
Servicio de autenticación	Python, FastAPI, tokens JWT	Guardar cuentas y roles, verificar contraseñas, emitir y validar tokens, gestionar sesiones y registrar eventos de autenticación.	CU-01, CU-02, CU-07, CU-09

Datos estadísticos medibles

Cada caso de uso principal genera datos que se pueden contar o medir. La Tabla 12 recoge los indicadores que pensamos calcular, el componente que registra cada dato y la forma de obtenerlo.

Tabla 12*Indicadores estadísticos generados por las operaciones principales*

Indicador	Casos de uso	Lo registra	Cómo se calcula
Inicios de sesión exitosos y fallidos por día	CU-01	Servicio FastAPI	Conteo de eventos por tipo y por fecha.
Tiempo de respuesta de la autenticación	CU-01	Servicio FastAPI	Diferencia, en milisegundos, entre la petición recibida y la respuesta enviada.
Duración de las sesiones	CU-01, CU-02, CU-09	Servicio FastAPI	Hora de cierre o vencimiento menos hora de inicio.
Mensajes por sala y por día	CU-03, CU-04	Servidor de chat	Conteo de mensajes agrupados por sala y fecha.
Latencia de entrega	CU-03	Servidor de chat	Tiempo entre la recepción del mensaje y su envío al último destinatario conectado.
Usuarios activos por día	CU-03, CU-04	Servidor de chat	Cantidad de usuarios distintos que enviaron al menos un mensaje.
Consultas de historial y su tiempo de respuesta	CU-05	Servidor de chat	Conteo de consultas y promedio de su duración.
Pico de usuarios conectados	CU-06	Servidor de chat	Máximo de conexiones activas simultáneas en el día.

Referencias

- Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.
- Donovan, A. A. A., y Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.
- Fette, I., y Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. <https://doi.org/10.17487/RFC6455>
- ISO/IEC/IEEE. (2018). *Systems and software engineering: Life cycle processes: Requirements engineering* (ISO/IEC/IEEE 29148:2018).
- Jones, M., Bradley, J., y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- Object Management Group. (2017). *Unified Modeling Language* (Versión 2.5.1). <https://www.omg.org/spec/UML/2.5.1/>
- Ramírez, S. (s. f.). *FastAPI*. <https://fastapi.tiangolo.com/>
- Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson.